Graylog Cluster OS Patch Otomasyonu

AAP + Bitbucket Kurulum ve Kullanım Kılavuzu

OpenSearch · MongoDB replica set · Graylog — rolling OS patch (os-patch-automation)

Sürüm	**v1.9.3**
Tarih	**02.11.2026**
Gizlilik	**Genel**
Paket	**AAP + Bitbucket**

İçindekiler

Genel Bakış 3

Doküman kontrolü 3

Amaç 3

Kapsam 4

Topoloji ve kullanılan servisler 4

Nasıl kullanılır 6

Roller ve sorumluluklar 7

Bu sürümün kılavuz seti 7

1. AAP + Bitbucket Kurulum ve Kullanım 8

Paket 8

Ön koşullar 8

Kurulum (bir kez) 9

C) İlk çalışma 11

D) Gece schedule'ı 11

E) Çalışırken ve sorun olursa 12

F) Yeni sürüm 12

Genel Bakış

Bu bölüm kılavuzun başında projenin amacını, kazanımlarını, kapsamını, hangi servislerle çalıştığını (topoloji) ve nasıl kullanıldığını özetler. AAP + Bitbucket kurulum ve kullanım bölümü bu bölümden sonra başlar.

Doküman kontrolü

Alan	Değer
Proje	Graylog / OpenSearch OS Patch Otomasyonu (graylog-patch-automation)
Doküman	Graylog Cluster OS Patch Otomasyonu — AAP + Bitbucket Kurulum ve Kullanım Kılavuzu
Doküman sürümü	**1.9.3** — proje sürümüyle aynıdır; kılavuz her proje sürümüyle yeniden üretilir
Yayın tarihi	02.11.2026
Kaynak	Proje deposu `graylog-patch-automation`, tag `v1.9.3` (release ekleri: paketler, `.sha256`, kılavuzlar)
Teslim paketleri	`os-patch-automation-1.9.3-terminal.tar.gz` · `os-patch-automation-1.9.3-aap.tar.gz`
Hazırlayan	Altyapı Otomasyonu

Amaç

Graylog / OpenSearch OS Patch Otomasyonu, Graylog log platformunu oluşturan **Graylog, OpenSearch ve MongoDB cluster sunucularına** işletim sistemi yamalarını (`yum update`, gerekiyorsa reboot) **sunucu sunucu (rolling)**, **okuma/yazma kesintisi olmadan** ve **bakım penceresi içinde** uygulayan bir Ansible otomasyonudur.

Her sunucuda servisler doğru sırada durdurulup başlatılır (durdurma: Graylog → MongoDB → OpenSearch; başlatma: MongoDB → OpenSearch → Graylog), rol taşıyan sunucular (OpenSearch aktif master, MongoDB PRIMARY, Graylog static leader) en sona bırakılır ve her adım doğrulanmadan sonrakine geçilmez. Hata olursa o sunucu geri alınır/toparlanır ve diğer sunuculara dokunulmaz. Aynı proje terminalden (offline Ansible makinesi) ve kurumun AAP + Bitbucket altyapısından çalışır.

Kazanım	Açıklama
Kesintisiz rolling patch	Yük altında 3 node rolling patch + reboot: **448–481 sn**, yazma ve GET okuma hatası **0**, veri kaybı 0, cluster hiç RED olmadı (gerçek OpenSearch 2.19.1 cluster'ı, systemd / yum / reboot benzetimli lab; üç koşu). Aramada yalnızca node'un kapandığı andaki kısmi sonuç ≤ %0,1; `strategy: drain` ile o da 0.
Güvenli geri alma	Geri alma senaryosu yük altında doğrulandı (187–225 sn): hata veren node'da `yum history undo`, node cluster'a döndü, diğer node'lara dokunulmadı, cluster GREEN, hata maili; okuma/yazma hatası 0, onaylanan tüm yazmalar mevcut.
AAP + Bitbucket ile doğrulandı	Gerçek 3 node Graylog cluster'ında (MongoDB 8.0 + OpenSearch 2.19.6 + Graylog 7.2): Bitbucket → AAP/AWX workflow (Checklist → Onay → Patch), 3 node sırayla reboot, toplam **751 sn**, node kesintisi 84–131 sn.
Bakım penceresine uyar	Her node'dan önce tahmini süre kalan pencereyle karşılaştırılır; sığmayan node'a başlanmaz, çalışma `PARTIAL` biter ve sonraki pencere için `only_nodes=...` komutu mailde verilir.
Denetlenebilir	Checklist (PLAN) ve sonuç için kurumsal HTML/JSON/Markdown rapor + mail (değişiklik kaydı numarası, onaylayan, node bazında adım saatleri ve kesinti süreleri, patch'lenen / ertelenen paketler).
Offline, ek bileşensiz	Yalnızca ansible-core (`ansible.builtin`); collection, container / EE imajı, podman veya ek sunucu gerekmez. Paketler Python 3.12 ve ansible-core wheel'lerini taşır.

Kapsam

Kapsamda

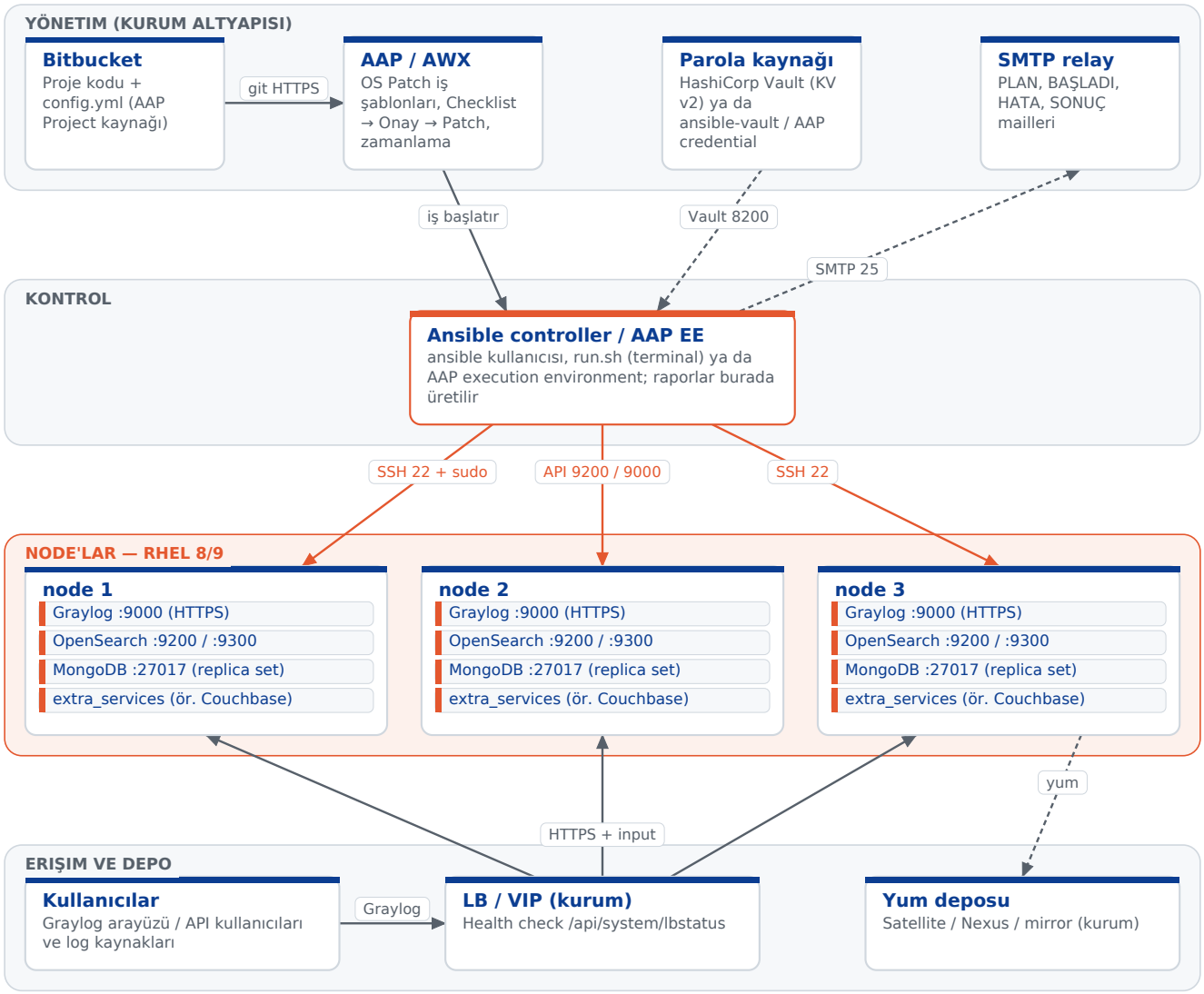
- İşletim sistemi paket güncellemesi: `yum update` (exclude politikası, `--skip-broken` → `--nobest`), kernel isteğe bağlı (`include_kernel`), reboot politikası `if_required` / `always` / `never`
- OpenSearch 2.x (Elasticsearch 7.x/8.x da desteklenir): `allocation.enable=primaries`, `_flush`, aktif master devri (voting exclusion), GREEN bekleme; `primaries` ya da `drain` stratejisi
- MongoDB replica set: sağlık, gecikme ve oplog kontrolü, PRIMARY'de `replSetStepDown`, PRIMARY en son
- Graylog: ALIVE / journal / cluster kontrolü, LB'den çıkarma (`lbstatus` DEAD → ALIVE), static leader en son
- Sunucu kontrolleri (disk %80, kernel img, SELinux, yum deposu), patch öncesi/sonrası OS ve servis durumu karşılaştırması ve düzeltme (route, mount, sysctl, swap, servisler)
- Bakım penceresi, hata durumunda geri alma (`yum history undo`), kontrollü durdurma (STOP) ve toparlama (`recover.yml`)
- Aynı sunucudaki diğer servisler (`extra_services`, ör. Couchbase): yalnızca `systemctl stop` / `start`
- AAP kurulumu (`tools/aap.sh` + `OS Patch | Yonetim | AAP Setup`), onaylı workflow, zamanlama (varsayılan kapalı)

Kapsam dışı

- Graylog, MongoDB ve OpenSearch **sürüm yükseltmesi** — bu paketler her zaman hariç tutulur (ayrı prosedür / Graylog Cluster Kurulumu projesi)
- VM snapshot ve servis yedekleri (OpenSearch snapshot, MongoDB dump, Graylog konfigürasyonu) — operatör alır; checklist'te onaylanır
- Yum deposunun (Satellite / Nexus / mirror) kurulumu ve içeriği; eski RPM'ler depoda yoksa geri alma çalışmaz
- Değişiklik kaydını açma/kapatma, monitoring bakım modu — operatör yapar
- `extra_services` servislerinin dosya, paket ve yapılandırması

Topoloji ve kullanılan servisler

Kod ve `config.yml` Bitbucket'ta durur; işler AAP'den (AAP execution environment) ya da terminal paketinde Ansible controller'dan çalışır. Controller node'lara `vtmroot` ile SSH + sudo ile bağlanır, OpenSearch ve Graylog API'lerini HTTPS ile sorgular; MongoDB komutları node üzerinde `mongosh` ile yerel çalışır. Node'lar paketleri kurumun yum deposundan alır. Lab ve küçük ölçekte üç servis aynı node'dadır; servisler ayrı sunucularda da olabilir (`nodes[].services`).



Şekil 1 — OS patch otomasyonu — yönetim katmanı, controller, cluster node'ları ve kullanıcılar

- Patch sırası: önce OpenSearch'süz (Graylog / MongoDB) node'lar — MongoDB PRIMARY en sonda —, sonra OpenSearch node'ları — aktif master en sonda; Graylog static leader devredilemediği için en son. Aynı anda yalnızca bir node durur (`serial: 1`).
- Node başına: Graylog stop (isteğe bağlı önce LB'de DEAD) → MongoDB (PRIMARY ise stepDown) stop → OpenSearch allocation/flush/stop → `yum update` → reboot (gerekirse) → MongoDB → OpenSearch → Graylog start → GREEN + ALIVE → sonraki node.
- Gerekli erişim: controller'dan node'lara TCP 22, 9200, 9000; SMTP 25; (varsa) Vault 8200. Node'lardan yum deposuna. AAP'den Bitbucket'a HTTPS.

Servisler, portlar ve bağlantılar

Servis	Görev	Port / protokol	Nerede
OpenSearch 2.x	Log index'leri. Patch'te `allocation.enable=primaries`, `_flush`, master devri; node dönünce allocation açılır, GREEN beklenir	9200 HTTPS (controller → node API) · 9300 transport (node'lar arası)	OpenSearch node'ları
MongoDB 7.0 / 8.0	Graylog yapılandırması (replica set). Gecikme / oplog kontrolü, PRIMARY'de `replSetStepDown`, PRIMARY en son	27017 TLS (node üzerinde `mongosh`, 127.0.0.1)	MongoDB node'ları
Graylog 6.x / 7.x	Log alma ve arama. `lbstatus` DEAD / ALIVE, journal ve cluster kontrolü, tek leader doğrulaması	9000 HTTPS (controller → node API)	Graylog node'ları

Servis	Görev	Port / protokol	Nerede
extra_services	Patch'in yönetmediği servisler (ör. Couchbase): yalnızca stop / start, isteğe bağlı port doğrulaması	config'te tanımlı (ör. 8091, 11210)	Aynı node'lar
Ansible controller / AAP EE	Otomasyonu çalıştırır, raporu üretir ve mail atar (`ansible` kullanıcısı; node'lara `vtmroot` + sudo)	SSH 22, 9200, 9000 (çıkış)	Yönetim sunucusu / AAP
Yum deposu	OS paketlerinin kaynağı; preflight'ta her node'da `yum makecache`, erişilemezse hiçbir node'a dokunulmaz	HTTP(S) (node → depo)	Kurum (Satellite / Nexus / mirror)
HashiCorp Vault (isteğe bağlı)	Parolalar KV v2'den (token / AppRole); alternatif: ansible-vault ile şifreli `secrets/<cluster>.yml` ya da AAP credential	8200 HTTPS	Kurum
SMTP relay	PLAN / sonuç raporları (`mail.to`) ve BAŞLADI / HATA bildirimleri (`mail.alert_to`)	25	Kurum
Bitbucket · AAP / AWX	Kod ve config deposu (AAP Project) · iş şablonları, onaylı workflow, zamanlama	HTTPS	Kurum

Nasıl kullanılır

Tüm ayarlar tek dosyada: `config.yml` (cluster'lar, node'lar ve servisleri, bakım penceresi, exclude politikası, mail). Parolalar `secrets/<cluster>.yml` (ansible-vault), HashiCorp Vault ya da AAP credential'dan gelir. Terminalde komutlar `./run.sh` ile, AAP'de iş şablonlarıyla çalıştırılır; akış ikisinde de aynıdır.

1 Kurulum
Paket sha256 → kur.sh; AAP için bir kez aap.sh check / push / bootstrap

2 Config
config.yml: node'lar, servisler, pencere, mail; parolalar

3 Checklist (PLAN)
dry_run: ön kontroller + PLAN raporu; sunuculara dokunmaz

4 Onay
Terminal: EVET · AAP: workflow onay adımı (onaylayan rapora yazılır)

5 Rolling patch
Node node: durdur → yum → reboot → başlat → GREEN; rol taşıyan en son

6 Sonuç raporu
SUCCESS / PARTIAL / FAILED / CHECK_FAILED + mail

7 STOP
Kontrollü durdurma: çalışan node biter, sıradaki başlamaz

8 Geri alma / toparlama
Hatada o node geri alınır, diğerlerine dokunulmaz; recover.yml

İş	Terminal (Ansible)	AAP iş şablonu	Not
Checklist (PLAN)	`./run.sh patch.yml -e cluster=<c> -e dry_run=true`	`OS Patch | <cluster> | 1 Checklist`	Sunuculara dokunmaz; PLAN raporu + mail
Onaylı patch	`./run.sh patch.yml -e cluster=<c>` (ekranda EVET)	`OS Patch | <cluster> | Checklist -> Onay -> Patch`	Önerilen yol; kapsam, reboot, değişiklik kaydı sorulur
Patch (onay önceden)	`... -e patch_scope=all -e checklist_confirmed=true`	`OS Patch | <cluster> | 2 Patch`	Checklist önceden tamamlanmış olmalı
Kontrollü durdurma	`./run.sh stop.yml -e cluster=<c>`	`OS Patch | <cluster> | 8 Kontrollu durdur`	Kaldırmak: `-e clear=true` / `8b Durdurma isaretini kaldir`
Toparlama	`./run.sh recover.yml -e cluster=<c> -e start_services=true`	`OS Patch | <cluster> | 9 Recover`	Yarıda kalan çalışmanın cluster ayarlarını temizler
AAP nesneleri	`tools/aap.sh check` / `push` / `bootstrap` (bir kez)	`OS Patch | Yonetim | AAP Setup`	config.yml değişince Bitbucket'a push + AAP Setup
Zamanlama	cron (isteğe bağlı)	`OS Patch | <cluster> | Zamanlama`	Varsayılan kapalı (`aap.schedule`)

Durdurmak için AAP'de Cancel, terminalde `kill -9` kullanılmaz: geri alma adımları çalışmaz. Kontrollü durdurma (STOP) kullanılır.

Geri alma `yum history undo` ile yapılır; eski RPM'ler depoda yoksa çalışmaz, kernel / glibc downgrade'i Red Hat desteklemez. Asıl geri dönüş yolu operatör checklist'inde zorunlu olan **VM snapshot**'tır.

Roller ve sorumluluklar

Rol	Sorumluluk
Operatör (kurum sistem ekibi)	Yedekleri alır, değişiklik kaydını açar, checklist'i çalıştırır, onay verir, sonuç raporunu kontrol edip kayda ekler.
Onaylayan (değişiklik sahibi)	PLAN raporunu inceler, değişikliği onaylar; AAP'de workflow onay adımını verebilir.
Platform ekibi	AAP, Bitbucket, Vault, yum deposu ve SMTP erişimini sağlar; AAP Setup'ı çalıştırır.
Bilgilendirilenler	Uygulama ekibi ve izleme merkezi: PLAN, BAŞLADI, HATA ve sonuç maillerini alır.
Destek ekibi	Mesai dışı FAILURE alarmında nöbetçi; rapor ve `logs/` ile destek verir.

Bu sürümün kılavuz seti

Tüm kılavuzlar aynı proje sürümüyle (1.9.3) üretilir; dosya adları `graylog-patch-automation-1.9.3-kilavuz` önekiyle başlar.

Kılavuz	Dosya	Kimin için
1. AAP + Bitbucket Kurulum ve Kullanım	`graylog-patch-automation-1.9.3-kilavuz-aap-bitbucket.pdf — bölüm 1`	Operatör + platform ekibi

1. AAP + Bitbucket Kurulum ve Kullanım

os-patch-automation 1.8.7: AAP + Bitbucket çevrimdışı paket kılavuzu

31 Ekim 2026

Paket

`os-patch-automation-1.8.7-aap.tar.gz` (~127 MB) ile **Graylog cluster'ının** (OpenSearch, MongoDB replica set ve Graylog) OS patch'i AAP / AWX arayüzünden, sunucu sunucu (rolling) yapılır. Kaynak kod kurumun Bitbucket reposunda durur ve AAP Project oradan çeker. İnternet gerekmez.

Paket, `.sha256` dosyasıyla birlikte teslim edilir; kurulumdan önce bütünlüğü doğrulanır.

Node başına akış terminal paketiyle aynıdır:

1. Graylog stop.
2. MongoDB stepDown (node PRIMARY ise) ve stop.
3. OpenSearch: allocation=primaries, flush, stop.
4. `yum update` ve reboot.
5. Servisler MongoDB → OpenSearch → Graylog sırasıyla başlatılır.
6. GREEN beklenir.

Ayrıntı: `docs/KILAVUZ_ANSIBLE_MAKINESI.md`. Rol taşıyan node'lar (static Graylog leader, OpenSearch aktif master, MongoDB PRIMARY) en son patch'lenir.

Paketteki	Ne işe yarar
`kur.sh`	Projeyi `/home/ansible/opensearch-patch-automation`'a koyar, ansible-core'u venv'e kurar (sistemde Python 3.9+ yoksa paketteki kullanır). Podman / container gerekmez
`ansible/python/`	Python 3.12 (yalnız sistemde 3.9+ yoksa kullanılır)
`proje/os-patch-automation/`	Bitbucket'a gönderilecek proje: `config.yml`, playbook'lar, `aap/`
`tools/aap.sh`	Yalnız ilk kurulum: `check` · `push` (Bitbucket, `git` ile) · `bootstrap` (AAP ilk kurulum) · `run`. Ansible makinesinde yerel ansible-core ile çalışır
`KILAVUZ.pdf`	Kılavuzların PDF hali
`SHA256SUMS.aap`	Paket içi dosya doğrulaması

Ön koşullar

Ne	Gereken
AAP / AWX	AAP 2.4+ (controller API `/api/v2`) veya AAP 2.5+ (`api_prefix: /api/controller/v2`), ya da AWX
Execution Environment	Özel imaj **yok**. AAP'de zaten tanımlı bir EE kullanılır (`config.yml > aap.execution_environment`, ör. `Default execution environment`). Playbook'lar yalnız `ansible.builtin` kullanır; collection / pip gerekmez. RHEL 8 hedefler için ansible-core ≤ 2.16 içeren EE (ör. `ee-minimal-rhel8` / `ee-supported-rhel8`)
Kurulum makinesi	Bitbucket'a ve AAP API'sine erişen herhangi bir Linux, `ansible` kullanıcısı, `git` (yalnız push için; yerel RHEL deposundan). Podman gerekmez
Ağ	Execution node'dan node'lara 22 / 9200 / 9000 ve SMTP 25 · AAP'den Bitbucket'a HTTP(S) · kurulum makinesinden Bitbucket'a ve AAP API'sine
Bitbucket	Proje (ör. `OPS`) ve repo (`os-patch-automation`; yoksa `create_repo: true` ile oluşturulur). Gönderim için yazma, AAP için okuma yetkili kullanıcı / HTTP access token
AAP Machine credential	`vtmroot-ssh` (`config.yml > aap.machine_credential`): kullanıcı `vtmroot`, SSH private key, Privilege Escalation `sudo` (+ parola gerekiyorsa). **AAP'de önceden oluşturulur**
Node'lar	Terminal kılavuzundakiyle aynı: RHEL 8/9, `mongosh`, MongoDB TLS CA'sı, yerel yum deposu

Kurulum (bir kez)

```
sha256sum -c os-patch-automation-1.8.7-aap.tar.gz.sha256
tar xzf os-patch-automation-1.8.7-aap.tar.gz && cd os-patch-automation-1.8.7-offline
sudo ./kur.sh                        # proje -> /home/ansible/opensearch-patch-automation,
    ansible-core -> venv
sudo -iu ansible ; cd ~/opensearch-patch-automation
```

Execution Environment: AAP'ye ek / özel imaj yüklenmez, EE oluşturulmaz. AAP'de zaten tanımlı EE'ler adla kullanılır:

Ayar	Ne için	Gereken
`aap.execution_environment`	Checklist / Patch / Recover / Durdur job'ları	ansible-core ≤ 2.16 (RHEL 8 hedefler). AAP: `Default execution environment` (ee-supported-rhel8)
`aap.management_execution_environment`	AAP Setup job'u (boş: yukarıdaki)	`ansible.controller` collection'ı (ee-supported'da var; AWX'te `awx.awx`, awx-ee'de var)

Ad AAP'de yoksa `check` uyarır, `bootstrap` durur.

1. config.yml

Terminal ve AAP **aynı** `config.yml`**'yi** kullanır. Cluster tanımı terminal kılavuzundaki gibi yapılır (`clusters:`, `nodes[].services`, mail, bakım penceresi). AAP'ye özel iki bölüm vardır:

```
global:
  scm:                                  # Bitbucket
    url: "https://bitbucket.<kurum>/scm/ops/os-patch-automation.git"
    branch: main
    username: <token sahibi kullanıcı>     # HTTP access token için: x-token-auth
    validate_certs: true
    ca_file: ""                         # kurumsal CA (PEM)
    create_repo: true                   # repo yoksa REST API ile oluştur
    project_name: OPS
  aap:
    url: "https://<aap>"
    api_prefix: /api/v2                 # AAP 2.5+: /api/controller/v2
    username: admin
    organization: Default
    execution_environment: "Default execution environment"   # AAP'de mevcut EE adı
    management_execution_environment: ""   # boş: aynı EE (ansible.controller içermeli)
    machine_credential: vtmroot-ssh        # AAP'de ÖNCEDEN oluşturulur
    clusters: [prd-graylog]                # job template + workflow kurulacak cluster'lar
    schedule: {enabled: false, ...}        # gece çalışması (bkz. D)
```

AAP'de cluster tanımına `ssh: {user: "", private_key_file: ""}` yazın. Böylece SSH bilgisi Machine credential'dan gelir.

2. Parolalar

```
cp secrets/example.yml secrets/prd-graylog.yml && vi secrets/prd-graylog.yml && chmod 600
    secrets/prd-graylog.yml
```

AAP'de dosya okunmaz. `bootstrap`, dosyadaki değerleri AAP'de `os-patch-prd-graylog-secrets` adlı credential'a (tip: `OS Patch Secrets`) bir kez aktarır. Sonra parola değişikliği **AAP arayüzünden** yapılır: Credentials → `os-patch-<cluster>-secrets` → Edit. `secrets/` Bitbucket'a **gönderilmez**. Sonradan eklenen cluster için credential AAP arayüzünden aynı adla ve tiple oluşturulur.

AAP Vault credential. `config.yml` içinde `!vault |` ile şifrelenmiş değerler (ör. `secrets.backend: inline` parolaları, SMTP parolası) Bitbucket'a şifreli gider. Job'larda bunları AAP'nin **Vault** credential'ı çözer:

```
printf '%s' 'PAROLA' | ansible-vault encrypt_string --vault-password-file ~/.ansible/os-patch-vault
    --stdin-name http_password   # çıktıyı config.yml'ye
AAP_VAULT_PASSWORD="$(cat ~/.ansible/os-patch-vault)" ./tools/aap.sh bootstrap     # AAP'de
    "os-patch-vault" (tip: Vault) oluşur, tüm job template'lere bağlanır
```

Vault parolası yalnızca AAP'de saklanır. Credential adı `aap.vault_credential` (varsayılan `os-patch-vault`). AAP arayüzünden elle oluşturulduysa AAP Setup onu adla job template'lere bağlar (`config.yml`'de `!vault` varsa). `secrets/<cluster>.yml` dosyası ansible-vault ile şifreliyse `bootstrap` dosyayı

`ANSIBLE_VAULT_PASSWORD_FILE` ile çözüp `OS Patch Secrets` credential'ına aktarır.

3. İlk kurulum (ansible makinesinden, bir kez)

```
./tools/aap.sh check        # ansible-core, syntax, config.yml, AAP API + EE adları, git, Bitbucket
    erişimi
./tools/aap.sh push         # proje -> Bitbucket, git ile (token / parola gizli sorulur: GL_SCM_TOKEN)
./tools/aap.sh bootstrap    # AAP ilk kurulum + "AAP Setup" job'unu AAP'de çalıştırır (AAP_PASSWORD
    veya AAP_TOKEN)
```

`push` önce Bitbucket'taki dalın geçmişini alır ve commit'i onun üzerine atar (force push yok). Esas olan push yapılan dizindeki dosyalardır; Bitbucket arayüzünden yapılan bir config değişikliği, bir sonraki push'ta bu dizindeki config ile ezilir. `config.yml` içinde `!vault` değerleri varsa vault parolası verilmelidir: `ANSIBLE_VAULT_PASSWORD_FILE=~/.ansible/os-patch-vault ./tools/aap.sh push` (terminalde verilmezse sorulur). Parola ve token'lar komut satırına, dosyaya veya ekrana yazılmaz. Ortamdan da verilebilir: `GL_SCM_TOKEN=... AAP_PASSWORD=... ./tools/aap.sh bootstrap`. Komutlar tekrar çalıştırılabilir: var olan nesne güncellenir, olmayan oluşturulur.

Kim oluşturur	Nesne	Ad
`bootstrap` (ansible makinesi, API)	Source Control credential	`os-patch-bitbucket`
`bootstrap`	Credential tipi + credential	`OS Patch Secrets` / `os-patch-<cluster>-secrets`
`bootstrap`	Vault credential	`os-patch-vault` (`AAP_VAULT_PASSWORD` verilirse; `config.yml` `!vault` değerleri)
`bootstrap`	Controller credential	`os-patch-controller` (tip: Red Hat Ansible Automation Platform; AAP Setup job'unun AAP API kimliği)
`bootstrap` + AAP Setup	Project / Inventory	`os-patch-automation` (Bitbucket, branch `main`) / `os-patch-localhost`
`bootstrap` + AAP Setup	Yönetim job template'i	`OS Patch | Yonetim | AAP Setup` (`aap/aap_setup.yml`, AAP modülleri)
AAP Setup (AAP içinde)	Job template'ler	`OS Patch | <cluster> | 1 Checklist` (dry-run) · `2 Patch` · `8 Kontrollu durdur` · `8b Durdurma isaretini kaldir` · `9 Recover`
AAP Setup	Workflow	`OS Patch | <cluster> | Checklist -> Onay -> Patch`
AAP Setup	Schedule	`OS Patch | <cluster> | Zamanlama` (`schedule.enabled: false` iken kapalı)

4. Sonrası: her şey AAP'de

Ansible makinesine bir daha gerek yoktur.

1. `config.yml` değişikliği Bitbucket'ta yapılır (web arayüzü veya git).
2. AAP → Templates → `OS Patch | Yonetim | AAP Setup` → Launch. Project'i Bitbucket'tan çeker ve job template'leri, workflow'ları, survey'leri ve zamanlamayı `config.yml`'ye göre kurar ya da günceller. Bunu AAP modülleriyle (`ansible.controller`), AAP'nin kendi EE'sinde yapar.
3. Parolalar AAP'de Credentials ekranından değiştirilir.

Saat dilimi: iş konteynerleri UTC çalışır. Bakım penceresi hesabı `maintenance_window.timezone` (Europe/Istanbul) ile yapıldığı için ek bir AAP ayarı gerekmez.

RPM / Nexus (isteğe bağlı). OS patch RPM gerektirmez. Node'larda eksik ön koşul paketi varsa RPM'ler `/home/ansible/setup/os-patch` dizininden veya Nexus'tan hazırlanır: `./tools/nexus.sh check | list | upload | download` (`config.yml > global.nexus`). Ayrıntı terminal kılavuzunda.

C) İlk çalışma

Checklist geçmeden ve onay verilmeden patch başlamaz.

```
OS Patch | <cluster> | Checklist -> Onay -> Patch
  1 Checklist ──başarılı──► Onay (operatör: backup'lar, CR, monitoring) ──onaylandı──► 2 Patch
```

1. **Checklist:** `Templates → OS Patch | <cluster> | Checklist -> Onay -> Patch → Launch` ile başlatılır. Survey'de şu sorular sorulur:
   - Patch kapsamı (`opensearch` = yalnızca OpenSearch sunucuları | `graylog` = Graylog + MongoDB sunucuları | `all`)
   - Yalnız bu sunucular (`only_nodes`, boş = kapsamdaki tümü)
   - İşlem (`os_patch` | `service_restart` = paket yok, yalnızca rolling servis restart)
   - Reboot (`if_required` = gerekirse, varsayılan | `always` = güncelleme olmasa da tüm node'lar sırayla reboot | `never`)
   - Değişiklik kaydı no (`change_id`, ör. CR-12345)
   - Bakım penceresi dışında çalıştır (`ignore_window`, varsayılan `false`)
2. Checklist hiçbir şeye dokunmaz. Sonuç PLAN raporu mail'le gelir; iş çıktısında ve **Artifacts** sekmesinde de görünür. Şunları kontrol edin: patch sırası ve roller, kontrol listesinde HATA olmadığı, patch'lenecek ve ertelenecek paketler.
3. **Onay:** `Views → Workflow Approvals → Approve`. Onaylayan kullanıcı rapora yazılır. Reddedilen onayda Patch çalışmaz.
4. **Patch** aynı kapsam ve `change_id` ile çalışır. Bitince sonuç raporu mail'le gelir.

Tek tek çalıştırmak için `1 Checklist` (dry-run) ve `2 Patch` şablonları kullanılabilir. Launch sırasında ek değişkenler verilebilir: `only_nodes`, `include_kernel`, `os_exclude`, `ignore_window`, `reboot_policy`.

Ne	Nereden (AAP 2.4; 2.5'te "Automation Execution" altında aynı adlar)
Checklist (dokunmaz)	Templates → `1 Checklist` → Launch
Patch (onaylı)	Templates → `Checklist -> Onay -> Patch` → Launch → survey
Onay	Views → Workflow Approvals → Approve / Deny
Kontrollü durdurma	Templates → `8 Kontrollu durdur` → Launch
Durdurma işaretini kaldırma	Templates → `8b Durdurma isaretini kaldir` → Launch
Toparlama	Templates → `9 Recover` → Launch (durmuş servisleri de başlatır)
Kodu / conf'u yenileme	Projects → `os-patch-automation` → Sync (her işte otomatik de yapılır)

D) Gece schedule'ı

`config.yml > global.aap.schedule` (cluster bazında `clusters.<cluster>.aap.schedule`):

```
schedule:
  enabled: false                # true + AAP'de "OS Patch | Yonetim | AAP Setup" ile açılır
  target: workflow              # workflow = Checklist -> Onay -> Patch | checklist = yalnızca kontrol
    + PLAN maili
  rrule: "DTSTART;TZID=Europe/Istanbul:20261101T000000 RRULE:FREQ=MONTHLY;INTERVAL=1;BYDAY=SU;BYSETPOS=1
    "
  patch_scope: all
  extra_vars: {}
```

- `target: workflow` olduğunda Onay adımı operatörü bekler. Tamamen onaysız gece patch'i istenmiyorsa bu önerilen yoldur.
- Başlangıç saati bakım penceresinin içinde olmalıdır. Pencere sınırı otomasyonun içinde de kontrol edilir.
- Açmak, kapatmak veya saati değiştirmek için `config.yml` Bitbucket'ta değiştirilir → AAP'de `OS Patch | Yonetim | AAP Setup` çalıştırılır.

E) Çalışırken ve sorun olursa

AAP'deki **Cancel düğmesini kullanmayın.** Ansible'ı anında keser ve geri alma adımları çalışamaz.

Durum	Ne yapılır
Kontrollü durdurmak	`8 Kontrollu durdur` çalıştırılır. Çalışan node biter, sonraki node'lar ertelenir, sonuç `PARTIAL` olur ve mail gider
Tekrar başlatmadan önce	`8b Durdurma isaretini kaldir`
İş yarıda öldü (pod silindi, Cancel)	`9 Recover`: cluster ayarlarını temizler, durmuş servisleri MongoDB → OpenSearch → Graylog sırasıyla başlatır
Patch hata verdi	Otomatik olarak: rollback denenir, servisler başlatılır, ayarlar geri alınır, hata maili (`alert_to`) gider, sonraki node'lara dokunulmaz
İş `ImagePullBackOff` ile başlamıyor	Seçili EE'nin imajı execution node'da / registry'de yok. AAP yöneticisi EE'yi kontrol eder ya da `aap.execution_environment`'a çalışan bir EE adı yazılıp AAP Setup tekrarlanır
`bootstrap`: "Execution Environment ... bulunamadi"	`aap.execution_environment` / `management_execution_environment` AAP > Execution Environments listesindeki adla birebir aynı olmalı
AAP Setup: "couldn't resolve module/action 'project'"	Yönetim EE'sinde `ansible.controller` / `awx.awx` yok. `aap.management_execution_environment`'a ee-supported (AAP) veya awx-ee (AWX) adını yazıp `bootstrap`'ı tekrarlayın
AAP Setup: "Controller credential bagli degil"	`os-patch-controller` credential'ı yönetim job template'ine bağlı değil. `bootstrap`'ı tekrarlayın
`push`: "git yok"	Yerel RHEL deposundan `sudo dnf install git`
Bitbucket push "License limit exceeded"	Bitbucket lisansı / kullanıcı limiti dolmuş. Kurum Bitbucket yöneticisine bildirin
AAP project sync "No default branch"	Boş repoya ilk push. `./tools/aap.sh push` (1.8.7+) varsayılan dalı kendisi ayarlar

F) Yeni sürüm

1. Yeni paketi açın, `sudo ./kur.sh` çalıştırın (proje + ansible-core). `config.yml` ve `secrets/` korunur; yeni şablon `config.yml.yeni` olarak gelir.
2. `diff config.yml config.yml.yeni` ile yeni ayarlara bakın.
3. `./tools/aap.sh push`: kod Bitbucket'a gider, `config.yml` sizin dosyanızdır.
4. AAP'de `OS Patch | Yonetim | AAP Setup`: yeni job template'ler ve survey alanları kurulur.
5. Templates → `1 Checklist` ile kontrol edin.